GraphRAG for Multi-Hop Question Answering with Dense, Graph Neural, and PCST Retrieval

Nikhil Juluri **Chinmay Kawle**
Department of Computer Science
University of Illinois at Chicago
njulu@uic.edu ckawl@uic.edu

Abstract

Retrieval-augmented generation systems often treat retrieval as an independent nearest-neighbor search problem, even when answering requires assembling evidence from multiple related documents. This report describes a GraphRAG implementation for HotpotQA-style multi-hop question answering. The system loads each question with its local distractor context, chunks documents, embeds chunks with BGE, constructs question-local hybrid graphs, and compares dense retrieval, dense-guided PCST retrieval, GraphSAGE retrieval, dense–GraphSAGE fusion, and fusion-guided PCST retrieval. Evaluation uses 300 HotpotQA train examples and an LLM answer-generation protocol scored by exact match and token F1. Dense–GraphSAGE fusion obtains the best overall answer score among the evaluated modes, with 0.6833 exact match and 0.7580 F1, improving over dense retrieval on bridge questions while slightly reducing comparison-question performance. PCST variants are implemented and useful for studying graph-constrained evidence expansion, but the current LLM evaluation does not show an end-to-end gain over the fusion retriever.

1 Introduction

Multi-hop question answering requires retrieving evidence chains rather than isolated passages. In HotpotQA, a bridge question may require using one entity to find another supporting article, while a comparison question may require retrieving and contrasting two entities [Yang et al., 2018]. Standard dense retrieval can recover semantically similar chunks, but it does not explicitly model relationships among chunks from the same title, adjacent context, keyword overlap, or semantic neighborhoods.

This project studies a compact GraphRAG pipeline in which retrieval is performed over question-local context graphs. The implementation builds on retrieval-augmented generation (RAG) [Lewis et al., 2020], dense passage retrieval ideas [Karpukhin et al., 2020], and graph neural networks [Hamilton et al., 2017]. It evaluates whether graph-aware ranking and graph-constrained expansion improve downstream LLM answer quality.

The main contributions of the report are:

- a faithful description of the implemented HotpotQA preprocessing, chunking, dense retrieval, hybrid graph construction, Query-Aware GraphSAGE scoring, fusion retrieval, and PCST-style expansion modules;
- an end-to-end answer evaluation of five retrieval modes using identical 300-question LLM evaluation files; and
- an analysis separating bridge and comparison questions, where the fusion model improves bridge answer F1 over dense retrieval but does not dominate every category.

Preprint. Under review.

2 Related work

Retrieval-augmented generation combines an external retriever with a conditional generator so that generated answers are grounded in retrieved evidence [Lewis et al., 2020]. Sparse lexical retrieval, including BM25 [Robertson and Zaragoza, 2009], remains a strong baseline in many open-domain QA settings because exact term matching can recover rare entities. The current evaluated code path does not implement a BM25 baseline; the repository focuses on dense and graph-based variants, while retaining "FAISS-only" labels for dense nearest-neighbor retrieval.

Dense Passage Retrieval (DPR) showed that learned dense encoders can outperform traditional sparse retrieval for open-domain QA when trained with question–passage supervision [Karpukhin et al., 2020]. This project uses SentenceTransformers with the BGE encoder rather than a DPR-trained dual encoder, but follows the same scoring pattern: encode the query and candidate chunks, normalize vectors, and rank by inner product.

Graph retrieval methods add structure to candidate evidence. GraphSAGE aggregates local neighborhood information and supports inductive node classification [Hamilton et al., 2017]; in this project, a Query-Aware GraphSAGE model predicts supporting chunk probabilities on per-question graphs. Prize-Collecting Steiner Tree (PCST) methods select valuable nodes while penalizing traversal cost [Goemans and Williamson, 1995]. The implementation uses a greedy multi-seed PCST-style expansion rather than an exact PCST solver, so we describe it as PCST-inspired graph selection.

3 Methodology

3.1 Dataset and preprocessing

The loader in `graphrag_env/src/loading.py` uses the Hugging Face `hotpot_qa` dataset with the `distractor` configuration. Each returned example stores the question id, question, answer, type, difficulty level, supporting fact titles and sentence ids, and the local context documents. Context documents are represented as LangChain `Document` objects with metadata including title, question id, question type, context index, source, and a Boolean `is_supporting` field.

3.2 Chunking and embedding

The chunking module uses a `RecursiveCharacterTextSplitter` with chunk size 300, overlap 50, and separators ordered from paragraphs down to characters. Very short documents below the minimum text length are skipped. Each chunk receives a stable id of the form `question_id::title::chunk_index`, plus chunk index and text-length metadata.

The embedding module encodes chunks with `BAAI/bge-base-en-v1.5`. Embeddings are generated per question, converted to NumPy arrays, and normalized. Query encoding uses the BGE search prefix:

```
Represent this sentence for searching relevant passages:
```

Dense scores are cosine similarities implemented as inner products between normalized query and chunk vectors.

3.3 Dense retrieval

Dense retrieval ranks chunks independently:

$$s_{\text{dense}}(q, c_i) = \mathbf{e}(q)^\top \mathbf{e}(c_i), \tag{1}$$

where both embeddings are normalized. The top-k chunks are returned and their titles are deduplicated in rank order for support-title recall evaluation. In the application and evaluation labels this mode is called "FAISS-only retrieval", although the inspected core evaluation function computes exact dense inner products over the local candidate set; optional FAISS acceleration appears in the application service code.

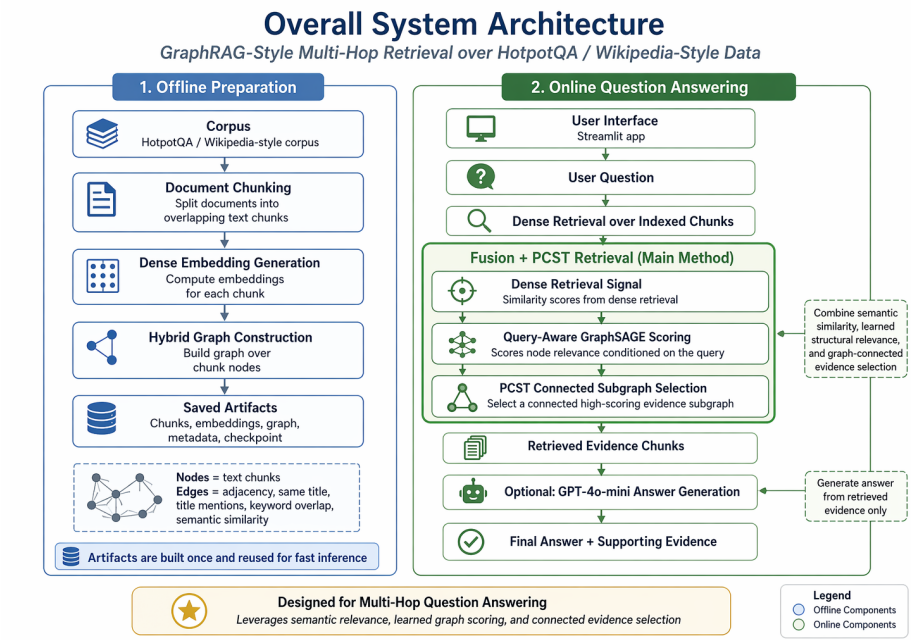

Figure 1: Implemented GraphRAG architecture. The system loads HotpotQA examples, chunks and embeds local context, builds hybrid graphs, and evaluates dense, GNN, fusion, and PCST-style retrieval before answer generation.

3.4 Sparse/BM25 retrieval

BM25 is relevant as a lexical baseline for multi-hop QA, but no BM25 or `rank_bm25` implementation is present in the inspected retrieval and evaluation modules. Therefore, this report does not include BM25 numbers. A future sparse or hybrid sparse–dense baseline could score chunks with BM25 and combine lexical and dense evidence as

$$s(q, c_i) = \alpha s_{\text{dense}}(q, c_i) + (1 - \alpha) s_{\text{BM25}}(q, c_i), \tag{2}$$

after appropriate normalization. This equation is included as a planned extension, not as an evaluated component.

3.5 Graph construction

The hybrid graph builder constructs one undirected NetworkX graph per question. Nodes correspond to chunks. Edges are added for four implemented relations: chunks from the same title, adjacent chunks within a title, semantic k-nearest-neighbor links using chunk embedding similarity, and keyword-overlap links after stopword filtering. The default graph settings in the evaluation scripts use semantic $k = 2$, semantic minimum similarity 0.40, and keyword-overlap threshold 3.

The cached 10,000-example artifact contains 263,113 global chunks. Across these question-local graphs, the average graph has 26.31 nodes and 106.69 edges, with 4.13 supporting chunk nodes on average. Edge construction is dominated by keyword-overlap and semantic-neighborhood relations: the artifact contains 794,131 keyword-overlap edge labels, 376,596 semantic-kNN labels, 348,207 same-title labels, and 165,605 adjacent-chunk labels. These statistics are useful because they show that the GNN and PCST modules operate over small local graphs rather than one monolithic corpus graph.

3.6 GNN-based retrieval

The GNN module implements a Query-Aware GraphSAGE model with two `SAGEConv` layers, batch normalization, ReLU activations, dropout, a residual connection after the first layer, and a linear

binary classifier. For each chunk node, the input feature vector concatenates the chunk embedding, repeated query embedding, and dense cosine similarity:

$$\mathbf{x}_i = [\mathbf{e}(c_i); \mathbf{e}(q); s_{\text{dense}}(q, c_i)]. \tag{3}$$

Labels are derived from the chunk metadata: supporting chunks are positive, all other local chunks are negative. Training uses binary cross-entropy with a global positive-class weight computed from the training split, Adam, gradient clipping, and early stopping on validation loss. GNN retrieval ranks chunks by sigmoid node probability.

3.7 Fusion and PCST retrieval

Fusion retrieval combines normalized dense scores and normalized GNN probabilities:

$$s_{\text{fusion}}(q, c_i) = \lambda s^{\text{norm}}_{\text{dense}}(q, c_i) + (1 - \lambda) s^{\text{norm}}_{\text{gnn}}(q, c_i). \tag{4}$$

The LLM evaluation uses $\lambda = 0.5$ for fusion and PCST modes.

Two PCST-style variants are implemented. Dense-guided PCST seeds the greedy expansion with dense scores. The main PCST mode first computes dense–GNN fusion scores, selects multiple high-prize seed nodes, expands through graph neighbors according to a prize-minus-edge-cost heuristic, preserves the top fusion anchors, adds a small PCST bonus to selected nodes, and applies a title-diversity bonus during final ranking. This is a practical, greedy approximation of the PCST objective rather than an exact solver.

4 Experimental setup

4.1 Data and retrieval settings

The LLM evaluation files cover 300 HotpotQA train examples, with 237 bridge questions and 63 comparison questions. Retrieval depth is top 5. The graph-aware scripts mirror the train/validation split from GNN training for retrieval recall experiments, while the LLM result JSON files used here report end-to-end answer quality for 300 examples per mode.

4.2 Answer generation and metrics

The evaluation script retrieves chunks, builds a context prompt from the top retrieved chunks, and asks `gpt-4o-mini` at temperature 0 to return a short JSON answer using only the provided context. If no API key is configured, the script contains a heuristic fallback answerer, but the result JSON files used in this report are the saved evaluation artifacts. Predictions are scored with HotpotQA-style normalized exact match (EM) and token F1. Metrics are reported overall and separately for bridge and comparison questions.

4.3 Inference timing protocol

We additionally measure retrieval-only inference time using cached artifacts. Timing excludes offline data loading, chunking, embedding generation for passages, graph construction, GNN training, model loading, and LLM answer generation. Each measurement includes the online query embedding and the retrieval computation required by that mode. We time 500 randomly sampled cached HotpotQA graph examples after warmup queries on the local arm64 development machine.

5 Results

Table 1 reports retrieval-side support-title recall from the repository's result-table generation script. Unlike the LLM answer table, the dense row uses all 10,000 examples, while the graph-aware rows use the shared 2,000-question held-out split created by the GNN pipeline. The main PCST variant has the highest support recall@5 in this retrieval-only evaluation, reaching 0.8045 overall and 0.8014 on bridge questions. This shows that PCST expansion can improve evidence-title coverage even when its downstream LLM answer score is weaker.

Table 1: Retrieval evaluation with support recall@5 (SR@5) and partial support recall@5 (P-SR@5). Dense is reported on 10,000 examples; graph-aware modes are reported on the 2,000-question held-out split.

Retriever	n	Overall SR@5	Overall P-SR@5	Bridge SR@5	Bridge P-SR@5	Comp. SR@5	Comp. P-SR@5
Dense (FAISS-only label)	10000	0.7752	0.9898	0.7618	0.9898	**0.8330**	**0.9899**
Dense-guided PCST	2000	0.7430	0.9855	0.7288	0.9852	0.8048	0.9866
GNN retrieval	2000	0.7675	0.9870	0.7589	0.9871	0.8048	0.9866
Dense + GraphSAGE fusion	2000	0.7940	0.9915	0.7897	0.9926	0.8128	0.9866
Dense + GraphSAGE + PCST	2000	**0.8045**	**0.9925**	**0.8014**	**0.9939**	0.8182	0.9866

Table 2: LLM answer evaluation on 300 HotpotQA examples. EM and F1 are computed after answer normalization.

Retriever	Overall EM	Overall F1	Bridge EM	Bridge F1	Comp. EM	Comp. F1
Dense (FAISS-only label)	0.6600	0.7338	0.6414	0.7216	**0.7302**	**0.7797**
Dense-guided PCST	0.6500	0.7193	0.6371	0.7117	0.6984	0.7480
GNN retrieval	0.6600	0.7380	0.6414	0.7297	**0.7302**	0.7692
Dense + GraphSAGE fusion	**0.6833**	**0.7580**	**0.6751**	**0.7564**	0.7143	0.7639
Dense + GraphSAGE + PCST	0.6500	0.7177	0.6329	0.7027	0.7143	0.7745

Table 2 reports answer-level metrics from the root-level JSON result files. Dense–GraphSAGE fusion is the best overall mode among the evaluated files, with 0.6833 EM and 0.7580 F1. The gain is concentrated on bridge questions: bridge F1 improves from 0.7216 for dense retrieval to 0.7564 for fusion. Comparison questions show the opposite pattern, where dense and GNN retrieval have higher EM than fusion, and dense retrieval has the highest comparison F1.

5.1 Bridge versus comparison behavior

Bridge questions benefit most from the learned fusion retriever. This matches the intended use of graph information: a GNN can use local structure and query-conditioned node features to promote chunks that are not merely individually similar to the query, while dense scores preserve strong semantic matches. The bridge improvement is modest but consistent across EM and F1 in the saved evaluation files.

Comparison questions are less clearly helped by graph expansion. Dense retrieval already tends to recover the two compared entities when the query names them explicitly, and graph-based reranking can introduce alternative titles that are structurally connected but less useful for direct comparison. The PCST main method has comparison F1 of 0.7745, close to dense retrieval, but its bridge degradation reduces overall performance.

6 Discussion

The strongest result is that simple score fusion is more reliable than replacing dense retrieval with a graph-only or graph-expanded ranking. Dense retrieval provides a robust semantic prior, while Query-Aware GraphSAGE contributes a learned estimate of support relevance. The implemented fusion equation is therefore an effective conservative combination.

The PCST-style methods are more mixed. Retrieval recall improves for the main PCST method, but answer quality drops relative to dense–GNN fusion. The code includes several safeguards: fallback filling when graph expansion stalls, preservation of top fusion anchors, a PCST bonus, and a title-diversity bonus. These choices make the method robust enough to return full result sets, but the qualitative examples suggest that graph expansion can still alter prompt composition in ways that move answer-critical context away from the generator or introduce confusable alternatives. This is especially costly when the generator receives only five chunks.

The figures and result table should also be interpreted as project-scale evidence rather than a benchmark claim. The evaluation uses 300 examples, one prompt template, one LLM, and one set of retrieval hyperparameters. The results are useful for guiding development, but not for claiming state-of-the-art performance.

Table 3: Retrieval-only inference time per query in milliseconds over 500 measured examples. Timing excludes model/artifact loading and LLM answer generation.

Retriever	n	Mean ms	P50 ms	P95 ms
Dense	500	19.85	17.01	38.44
Dense-guided PCST	500	**16.81**	16.15	25.45
GNN retrieval	500	16.97	16.15	**25.31**
Dense + GraphSAGE fusion	500	33.84	31.84	52.59
Dense + GraphSAGE + PCST	500	32.96	30.96	51.10

Retrieval Evaluation — HotpotQA (top-k = 5)

Method	Overall			Bridge			Comparison		
	n	SR@5	P-SR@5	n	SR@5	P-SR@5	n	SR@5	P-SR@5
Dense (FAISS-only)	10000	0.7752	0.9898	8120	0.7618	0.9898	1880	0.8330	0.9899
Dense-Guided PCST	2000	0.7430	0.9855	1626	0.7288	0.9852	374	0.8048	0.9866
GNN	2000	0.7675	0.9870	1626	0.7589	0.9871	374	0.8048	0.9866
Dense + GNN Fusion (λ=0.5)	2000	0.7940	0.9915	1626	0.7897	0.9926	374	0.8128	0.9866
Multi-Seed PCST (Ours ★)	2000	0.8045	0.9925	1626	0.8014	0.9939	374	0.8182	0.9866

Figure 2: Saved result visualization comparing answer EM and F1 across retrieval modes.

7 Limitations

The current report has several implementation and evaluation limitations. First, BM25 is discussed as related work and a future baseline, but it is not implemented in the inspected retrieval code. Second, the PCST implementation is greedy and PCST-inspired rather than an exact prize-collecting Steiner tree solver. Third, the LLM evaluation depends on one model and one prompt format, and the saved JSON metrics do not include confidence intervals. Fourth, the answer evaluation is run on a 300-example subset of the HotpotQA train split rather than the official development set. Finally, graph construction is question-local over HotpotQA distractor contexts; global corpus retrieval behavior may differ from these controlled local-context experiments.

8 Conclusion

This GraphRAG project implements a practical multi-hop QA retrieval stack with dense retrieval, hybrid graph construction, Query-Aware GraphSAGE scoring, dense–GNN fusion, and PCST-style graph expansion. On the saved 300-question LLM evaluation, dense–GraphSAGE fusion provides the best overall answer quality and the clearest bridge-question improvement. The graph and PCST modules remain valuable for evidence-chain experiments, but the current results indicate that graph expansion needs careful calibration when only a small number of chunks reach the generator.

Reproducibility notes

The paper can be compiled from the repository root with:

```
cd paper
pdflatex main.tex
bibtex main
pdflatex main.tex
pdflatex main.tex
```

The metrics in Table 2 are taken from `llm_eval_results_dense.json`, `llm_eval_results_fusion.json`, `llm_eval_results_gnn.json`, `llm_eval_results_pcst.json`, and `llm_eval_results_pcst_dense.json` at the repository root.

Retrieval Evaluation Results — HotpotQA (top-k = 5)

Overall (all question types)

Method	Overall		
	n	SR@5	P-SR@5
Dense (FAISS-only)	10000	0.7752	0.9898
Dense-Guided PCST	2000	0.7430	0.9855
GNN	2000	0.7675	0.9870
Dense + GNN Fusion (λ=0.5)	2000	0.7940	0.9915
Multi-Seed PCST (Ours ⋆)	2000	**0.8045**	0.9925

Bridge Questions

Method	Bridge		
	n	SR@5	P-SR@5
Dense (FAISS-only)	8120	0.7618	0.9898
Dense-Guided PCST	1626	0.7288	0.9852
GNN	1626	0.7589	0.9871
Dense + GNN Fusion (λ=0.5)	1626	0.7897	0.9926
Multi-Seed PCST (Ours ⋆)	1626	**0.8014**	0.9939

Comparison Questions

Method	Comparison		
	n	SR@5	P-SR@5
Dense (FAISS-only)	1880	0.8330	0.9899
Dense-Guided PCST	374	0.8048	0.9866
GNN	374	0.8048	0.9866
Dense + GNN Fusion (λ=0.5)	374	0.8128	0.9866
Multi-Seed PCST (Ours ⋆)	374	**0.8182**	0.9866

Figure 3: Tabular visualization of the LLM evaluation results included with the project assets.

References

Michel X. Goemans and David P. Williamson. A general approximation technique for constrained forest problems. *SIAM Journal on Computing*, 24(2):296–317, 1995.

William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781, 2020.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question

Table 4: Qualitative cases from the saved LLM evaluation JSONs. These examples illustrate why fusion improves bridge answers but graph expansion can still hurt downstream generation.

Case	Question summary	Gold	Baseline output	Alternative output
Fusion fixes dense	Nationality of James Henry Miller's wife	American	Dense: insufficient evidence	Fusion: American
Fusion fixes dense	Origin of the music played by Die Rhoner Sauwantzt	United States	Dense: Hessen, Germany	Fusion: United States
PCST hurts dense	Year of birth for the 2016 Marrakesh ePrix winner	1988	Dense: 1988	PCST: 1984
Comparison noise	Whether Robert Fleischman and Jimmy Barnes share nationality	no	Dense: No	Fusion: insufficient evidence

answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, 2018.

A NeurIPS checklist

1. For all authors: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope? [Yes] The report limits claims to the implemented pipeline and saved evaluation files.
2. Did you describe the limitations of your work? [Yes] See the Limitations section.
3. Did you discuss any potential negative societal impacts of your work? [N/A] This is a project report on retrieval methodology over an existing QA dataset.
4. Did you include complete proofs for theoretical results? [N/A] The report contains no theoretical claims requiring proofs.
5. Did you include the code, data, and instructions needed to reproduce the main experimental results? [Yes] The repository contains the code and saved result files; compile instructions are included above.
6. Did you specify all training details relevant to the reported results? [Yes] The report summarizes chunking, embedding, graph construction, GraphSAGE features, optimization choices visible in the code, and LLM evaluation settings.
7. Did you report error bars? [No] The saved JSON artifacts contain point estimates only.
8. Did you include the compute resources used? [No] The inspected result files do not record hardware details.
9. Did you describe dataset splits and preprocessing? [Yes] See Dataset and preprocessing and Experimental setup.